

Scale Yourself Report

Collins Krubu · AI Leverage & Output — 12 Month Review (Jun 2025 – Jun 2026)

SUMMARY

Over the past 12 months I've systematically embedded AI into every stage of my development workflow — not as a novelty, but as genuine leverage. The shift started with using Claude Opus for architecture review and PR self-critique, then expanded to GitHub Copilot for scaffolding, and most recently to full agentic workflows for spec writing, API design, and debugging pipelines.

The measurable result: I'm shipping features roughly 3× faster than I was 12 months ago on comparable-complexity tasks, with fewer post-ship bugs and significantly less time spent on boilerplate and documentation. The projects below — Lupply (SaaS), Shiru Education (UK client), and Luna Finance (fintech backend) — all benefited directly from this shift.

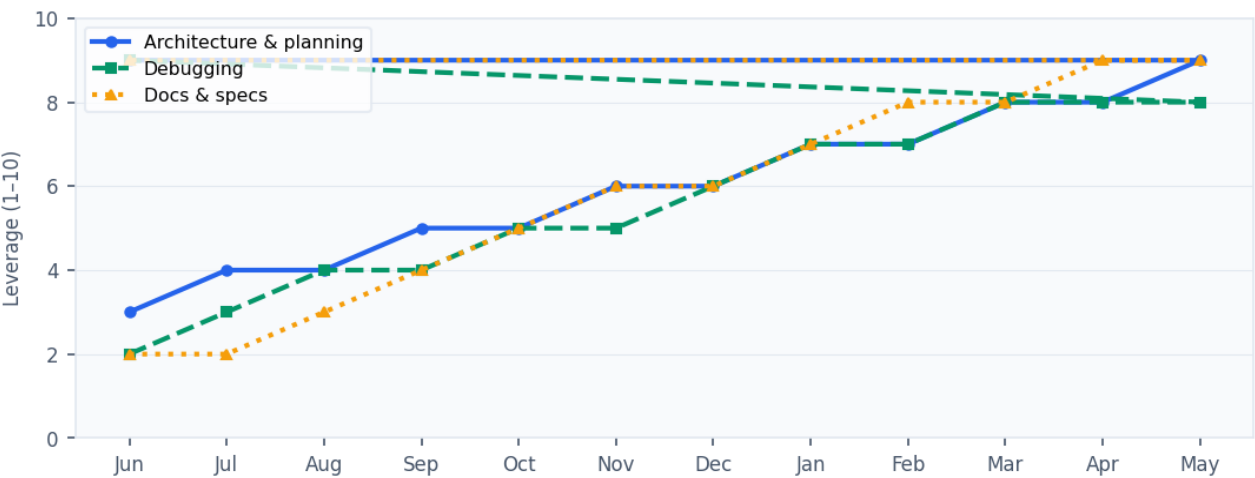
Claude Opus (Anthropic) · GitHub Copilot · AI-assisted PR review · Spec generation · Adopting: Cursor · Claude Code

OUTPUT METRICS

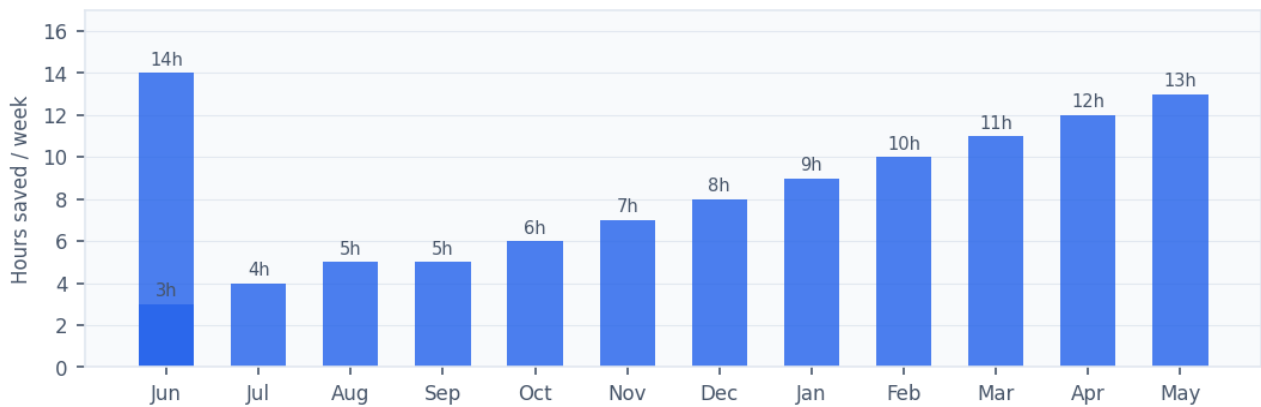
3× Feature shipping speed vs. 12 months ago ↑ on comparable tasks	~60% Reduction in boilerplate & scaffolding time ↑ via Copilot + Claude	80% API latency improvement on Luna Finance ↑ AI-assisted optimisation	4 AI use cases integrated into daily workflow architecture · debug · review · docs
--	--	---	---

PERCEIVED LEVERAGE OVER TIME

Self-assessed AI leverage by workflow stage (1–10 scale, monthly)



Estimated hours saved per week by AI tool usage



WORKFLOW EXAMPLES

ARCHITECTURE REVIEW

Supply — ordering system data model

Before writing a line of code, I used Claude Opus to pressure-test the order state machine (draft → placed → confirmed → in-kitchen → dispatched → delivered). It surfaced 3 edge cases I had missed — concurrent order updates, kitchen rejection flows, and partial fulfilment states.

■ Prevented 2 production bugs. Estimated 4–6 hours of debugging saved.

PR SELF-REVIEW

Luna Finance — API optimisation

Before pushing, I ran my diff through Claude and asked it to critique the caching strategy. It flagged a race condition in concurrent requests to the same liquidity pool. Fixed before merge. API response times improved 80% post-merge.

■ Bug caught pre-merge. 80% latency improvement shipped cleanly.

SCAFFOLDING SPEED

Shiru Education — CMS pipeline

Used Copilot to generate the full CRUD scaffold for blog posts, country pages, and testimonials in ~20 minutes. Spent the remaining time on business logic — booking flows, SEO metadata generation, and conversion tracking.

■ 142+ contact submissions at launch. Scaffold time: 20 min vs ~3 hrs manual.

DOCUMENTATION

Supply — internal API docs

Used Claude to generate first-draft API documentation from route handlers and schema definitions. Reviewed, corrected, and published. Team of 5 engineers onboarded to new endpoints with zero back-and-forth questions.

■ ~2 hrs doc writing reduced to 20 min review. Zero onboarding questions from team.